

PRÁCTICA 3 — METAHEURÍSTICAS

Optimización del Problema de Asignación Cuadrática (QAP) en la Planta Ford Valencia



Autores

Alumno 1: José Javier Torres Cantero

Correo: jjtc0005@red.ujaen.es

Alumno 2: David Fernández Espejo

Correo: dfe00004@red.ujaen.es

Grado en Ingeniería Informática

Asignatura: Metaheurísticas

Curso: 2025–2026

ÍNDICE

[1. Definición del Problema](#)

[1.1 Descripción](#)

[1.2 Representación de la solución](#)

[1.3 Ejemplo visual](#)

[2. Parámetros de ejecución](#)

[3. Descripción general del algoritmo memético](#)

[3.1 Inicialización y parámetros internos](#)

[3.2 Estructura evolutiva general y mantenimiento del mejor individuo](#)

[3.3 Disparadores meméticos por hitos de evaluaciones](#)

[3.4 Selección por torneos y efectos en la presión selectiva](#)

[3.5 Operadores evolutivos específicos utilizados](#)

- [Cruce OX2](#)

- [Mutación basada en intercambios 2-opt](#)

[3.6 Proceso de reemplazo y reinserción del elitismo](#)

[3.7 Integración interna de la Búsqueda Tabú](#)

[3.8 Progreso generacional y finalización de la ejecución](#)

[3.9 Pseudocódigo](#)

[4. Resultados](#)

[4.1 NumEvalTabu1000](#)

[4.1.1 NumIteraciones10](#)

[4.1.2 NumIteraciones50](#)

[4.1.3 NumIteraciones100](#)

[4.1.4 MejorNumEval1000](#)

[4.1.5 Gráfica](#)

[4.2 NumEvalTabu2000](#)

[4.2.1 NumIteraciones10](#)

[4.2.2 NumIteraciones50](#)

[4.2.3 NumIteraciones100](#)

[4.2.4 MejorNumEval2000](#)

[4.2.5 Gráfica](#)

[4.3 NumEvalTabu5000](#)

[4.3.1 NumIteraciones10](#)

[4.3.2 NumIteraciones50](#)

[4.3.3 NumIteraciones100](#)

[4.3.4 MejorNumEval5000](#)

[4.3.5 Gráfica](#)

[4.4 Comparación](#)

[4.4.1 Gráfica](#)

[4.5 Comparación final](#)

[4.5.1 Gráfica](#)

[5. Logs](#)

1. Definición del Problema

1.1 Descripción

El problema consiste en determinar la asignación óptima de departamentos a localizaciones dentro de la planta de montaje de Ford Valencia. Cada departamento envía y recibe un flujo de piezas respecto a los demás, y cada par de localizaciones tiene asociada una distancia.

El objetivo es **minimizar el coste total de transporte interno**, obtenido al multiplicar los flujos de piezas entre departamentos por las distancias correspondientes entre las localizaciones asignadas.

Para modelarlo, se utilizan dos matrices cuadradas:

- **F = (F_{ij})**: matriz de flujos, donde F_{ij} indica el número de piezas que se transportan del departamento *i* al *j*.
- **D = (D_{kl})**: matriz de distancias, donde D_{kl} representa la distancia entre las localizaciones *k* y *l*.

La función objetivo a minimizar es:

$$\min \sum_{i=1}^n \sum_{j=1}^n F_{ij} \cdot D_{S(i)S(j)}$$

donde **S** es una *permutación* que representa la asignación de cada departamento a una localización.

Este problema pertenece a la clase **NP-completa** y es conocido como el **Problema de Asignación Cuadrática (QAP)**, caracterizado por su alta complejidad computacional y su importancia en la optimización industrial.

1.2 Representación de la solución

En el Problema de Asignación Cuadrática (QAP), una solución se representa mediante un vector de enteros **S** de longitud *n*, donde *n* corresponde al número de departamentos y al mismo número de localizaciones disponibles. Cada posición del vector **S** identifica un departamento, y el valor almacenado en ella indica la localización asignada a dicho departamento.

El vector de solución tiene la forma:

$$S = [S_0, S_1, S_2, \dots, S_{n-1}]$$

El índice i representa el departamento i , mientras que el valor S_i indica la localización que le ha sido asignada. Una solución es factible cuando S contiene una permutación de los valores $\{0, 1, 2, \dots, n-1\}$, ya que cada localización sólo puede asignarse a un único departamento.

1.3 Ejemplo visual

Si se obtiene una solución $S = [2, 0, 3, 1]$, la interpretación es:

Departamento	Localización asignada
0	2
1	0
2	3
3	1

Cada par departamento–localización define una asignación válida dentro de la planta

D	L	Asignación ($s[i] = j$)
D0	L2	$S[0] = 2$
D1	L0	$S[1] = 0$
D2	L3	$S[2] = 3$
D3	L1	$S[3] = 1$

2. Parámetros de ejecución

En esta práctica se explicarán nuevamente todos los parámetros utilizados, ya que se combinan diferentes técnicas vistas anteriormente y es necesario detallar cada uno de ellos para comprender su función y cómo influyen en el comportamiento del proceso de búsqueda.

A continuación se describen todos los parámetros:

- **archivo**: indica la ruta al fichero que contiene la instancia del problema a resolver.
- **algoritmo**: especifica el nombre del algoritmo que se ejecutará.
- **k**: entero que determina el tamaño del vecindario o el número de elementos considerados en determinadas operaciones internas.
- **semillas**: lista de valores enteros utilizados para inicializar el generador de números aleatorios y garantizar la reproducibilidad de los experimentos.
- **iteraciones**: número máximo de iteraciones permitidas en las diferentes fases del proceso.
- **posSemilla**: entero que indica qué semilla de la lista se utilizará en la ejecución actual.
- **tenencia**: entero que define la longitud de la lista tabú, determinando cuántos movimientos quedan prohibidos temporalmente.
- **elitismo**: número de individuos de mayor calidad que se mantienen entre generaciones sin ser modificados.
- **kBest**: número de individuos con mejor evaluación que participan en los torneos de selección.
- **kWorst**: número de individuos con peor evaluación que también se incluyen en los torneos, aportando mayor diversidad.
- **operadorCruce**: cadena que indica qué operador de cruce se empleará para generar nuevos individuos.
- **probCruce**: probabilidad con la que se aplica el cruce durante la reproducción. Si no ocurre cruce, el descendiente se genera sin combinar información genética.
- **probMutacion**: probabilidad con la que se aplica la mutación sobre cada individuo generado.
- **evaluaciones**: límite máximo del número total de evaluaciones de la función objetivo permitidas.
- **tamPoblacion**: número total de individuos que componen la población en cada generación.

- **probGreedy**: porcentaje de la población inicial que se genera mediante un procedimiento GreedyRandom.
- **intervaloTabu**: Intervalo de evaluaciones entre activaciones de búsqueda Tabú sobre el individuo elite.
- **iteracionesTabu**: número de iteraciones de cada ejecución de búsqueda Tabú en el disparador memético

3. Descripción general del algoritmo memético

En esta sección se detalla el funcionamiento particular del algoritmo memético implementado, centrado en los mecanismos internos que lo diferencian de los algoritmos evolutivos convencionales y en la manera en que incorpora búsquedas locales intensivas (Búsqueda Tabú) a lo largo de la evolución. Estas explicaciones complementan lo descrito en los apartados anteriores, poniendo el foco en la integración dinámica de la Búsqueda Tabú dentro del proceso generacional y en las decisiones internas que caracterizan esta implementación concreta.

El algoritmo combina tres pilares fundamentales: **evolución poblacional**, **elitismo reforzado** y **activación periódica de búsqueda Tabú**. Esta interacción permite mantener diversidad, mejorar individuos clave y redirigir la exploración cuando el proceso se estabiliza o deja de progresar.

3.1 Inicialización y parámetros internos

Además de los parámetros comunes ya descritos en prácticas anteriores, esta implementación introduce varios elementos adicionales que controlan el comportamiento del módulo memético y la activación de búsquedas locales:

- **solucionActual** y **solucionAnterior**: mantienen tanto el individuo elite actual como su versión previa antes de aplicar Tabú. Esto permite revertir un intento fallido de intensificación si el proceso empeora la solución.
- **tenenciaTabu**: define la longitud de la lista tabú empleada específicamente en las fases meméticas internas, independiente de la utilizada en implementaciones externas. En este caso se mantiene un tamaño reducido para permitir una exploración intensiva sin comprometer la variación genética.
- **nGreedy**: número de individuos iniciales generados mediante el procedimiento GreedyRandom, que determina la proporción de la población inicial que parte de configuraciones guiadas en lugar de generarse aleatoriamente. Este valor depende directamente de probGreedy.
- **iteracionesTabu**: número de iteraciones que se ejecuta la Búsqueda Tabú cada vez que se activa un disparador memético. Este parámetro regula la profundidad de la intensificación sobre el individuo elite y el esfuerzo exploratorio aplicado durante cada activación.
- **intervaloTabu**: determina el número de **evaluaciones acumuladas tras las cuales se dispara la Búsqueda Tabú**. Es decir, cada vez que el contador evaluaciones alcanza o supera siguienteDisparioTabu, se ejecuta la Tabú sobre el individuo elite (solucionActual).

La población inicial, tras ser construida, se acompaña del coste asociado a cada individuo, y de inmediato se identifica el mejor de ellos. Este individuo se convierte en referencia tanto para la evolución poblacional como para las fases meméticas posteriores.

3.2 Estructura evolutiva general y mantenimiento del mejor individuo

El algoritmo sigue un ciclo generacional clásico:

1. Se seleccionan padres mediante **torneos de tamaño kBest**.
2. Se generan descendientes mediante **cruce (OX2)** y **mutación (intercambios 2-opt)**.
3. Se forma la nueva población.

La implementación incorpora **elitismo absoluto**: el mejor individuo global no solo se conserva, sino que se reinyecta explícitamente en la nueva población reemplazando siempre al peor individuo generado. Esto garantiza:

- Supervivencia del mejor individuo.
- Estabilización del proceso evolutivo.
- Preservación de un punto de referencia sólido para las fases meméticas.

Así, incluso si la población converge o deriva hacia regiones peores del espacio de búsqueda, nunca se pierde la mejor configuración encontrada hasta el momento.

3.3 Disparadores meméticos por hitos de evaluaciones

La característica más relevante del algoritmo es la activación programada de **Búsqueda Tabú** cada intervaloTabu evaluaciones:

- El disparador aplica la búsqueda Tabú solo sobre **solucionActual** (el individuo élite).
- La búsqueda Tabú se ejecuta durante iteracionesTabu iteraciones.
- Si no hay mejora, se restaura **solucionAnterior** y se aplica una segunda búsqueda controlada.
- Cada disparador imprime información sobre la evolución del mejor individuo y el coste alcanzado.

Esto permite sincronizar la intensificación memética con la maduración de la población:

- Al inicio, se preserva diversidad y se realizan mejoras locales moderadas.
- En fases intermedias y avanzadas, se refina la solución élite mediante intensificaciones periódicas.

3.4 Selección por torneos y efectos en la presión selectiva

La selección de progenitores se realiza mediante torneos de tamaño kBest. Este método ya ha sido explicado en secciones anteriores, por lo que aquí se destaca únicamente su comportamiento específico en esta implementación:

- al aplicarse dos veces por cada descendiente, se refuerza la probabilidad de que los mejores individuos se reproduzcan;

- la presión selectiva se mantiene constante independientemente del progreso del algoritmo;
- en combinación con el elitismo absoluto, esta estrategia dinamiza la convergencia sin necesidad de ajustar parámetros.

Este enfoque evita la deriva genética prematura al no excluir completamente a los individuos peores, que aún pueden ganar torneos por azar, preservando así la diversidad mínima necesaria.

3.5 Operadores evolutivos específicos utilizados

En esta implementación se utiliza una combinación básica pero robusta de operadores evolutivos.

- **Cruce OX2**

El operador OX2 mantiene la validez de la permutación y transfiere subsecuencias del progenitor base mientras rellena el resto de posiciones respetando el orden relativo del segundo progenitor. Este mecanismo produce descendientes equilibrados entre explotación y exploración al conservar parcialmente la estructura de ambos padres.

- **Mutación basada en intercambios 2-opt**

La mutación se realiza intercambiando dos posiciones aleatorias del individuo. Aunque sencilla, esta mutación:

- genera una perturbación mínima pero significativa;
- mantiene la validez de la permutación;
- favorece la exploración de vecinos cercanos sin romper el proceso evolutivo.

Dado que las búsquedas Tabú internas aplican 2-opt de forma intensiva, esta mutación complementa la dinámica del algoritmo sin solapar su efecto.

3.6 Proceso de reemplazo y reinserción del elitismo

Una vez generada la nueva población, se evalúan los descendientes y antes de finalizar la generación se lleva a cabo un reemplazo explícito:

- se identifica el peor individuo generado;
- se sustituye por una copia del mejor global;
- se actualizan los costes en consecuencia.

Este mecanismo asegura que:

- la calidad mínima de la población no decrece;
- los operadores evolutivos no destruyen el progreso acumulado;
- la presión hacia la mejora se mantiene estable a lo largo del proceso evolutivo.

3.7 Integración interna de la Búsqueda Tabú

La Búsqueda Tabú utilizada aquí ha sido descrita ya con detalle en la sección correspondiente, por lo que solo se comentan aspectos específicos de su integración con el algoritmo memético general:

- La búsqueda se ejecuta directamente sobre `solucionActual`, sin necesidad de duplicar o reconstruir individuos.
- La implementación utiliza una lista tabú circular con movimientos simples 2-opt, lo que permite actualizaciones eficientes.
- Se calcula el delta de coste mediante una fórmula incremental que evita evaluaciones completas y acelera significativamente la búsqueda.
- Si no se detecta mejora tras aplicar Tabú con una cierta profundidad, se utiliza `solucionAnterior` como mecanismo de recuperación inmediato para evitar pérdidas de calidad.
- Cada fase Tabú imprime información detallada sobre el progreso, permitiendo verificar la evolución del mejor individuo durante la fase de intensificación.

Estas decisiones de diseño permiten que el módulo memético actúe como un mecanismo de refinamiento potente sin penalizar el rendimiento global del algoritmo evolutivo.

3.8 Progreso generacional y finalización de la ejecución

En cada generación se muestra por pantalla el número de iteraciones generadas, el número de evaluaciones acumuladas y el mejor coste global alcanzado. Esto permite monitorizar la evolución del algoritmo y analizar visualmente su convergencia.

Finalmente, al alcanzar el límite de evaluaciones, el proceso se detiene y se devuelve el mejor individuo global, que incorpora no solo la evolución genética previa sino también las mejoras aportadas por las fases sucesivas de intensificación memética.

3.9 Pseudocódigo

```
None
MEMETICO(instancia, tamPoblacion, kBest, kWorst, elitismo,
          operadorCruce, probCruce, probMutacion, probGreedy,
          maxEvaluaciones, tenenciaTabu, intervaloTabu, iteracionesTabu):

    # Paso 1. Inicialización de población
    P ← matriz de tamPoblacion individuos
    costes ← array de tamPoblacion
    evaluaciones ← 0
```

```

siguienteDisparoTabu ← intervaloTabu

nGreedy ← tamPoblacion * probGreedy

for i = 0 to tamPoblacion - 1:
    si i < nGreedy:
        P[i] ← construir_GreedyRandom(instancia)
    sino:
        P[i] ← generar_permutación_aleatoria()

    costes[i] ← evaluar(P[i])
    evaluaciones ← evaluaciones + 1

mejor ← mejor_individuo(P)
mejorCoste ← evaluar(mejor)
solucionActual ← copia(mejor)
solucionAnterior ← copia(mejor)

# Paso 2. Bucle evolutivo generacional
while evaluaciones < maxEvaluaciones:

    # Paso 2.1. Activación de disparador Tabú
    si evaluaciones ≥ siguienteDisparoTabu:
        # Aplicar Búsqueda Tabú sobre el individuo elite
        mejorado ← busquedaTabuModificada(solucionActual, iteracionesTabu,
tenenciaTabu)

        si no mejorado:
            # Recuperar solución anterior y reintentar Tabú
            solucionActual ← copia(solucionAnterior)
            busquedaTabuModificada(solucionActual, iteracionesTabu,
tenenciaTabu)

        # Actualizar mejor global si hubo mejora
        costeActual ← evaluar(solucionActual)
        si costeActual < mejorCoste:
            mejor ← copia(solucionActual)
            mejorCoste ← costeActual

        siguienteDisparoTabu ← siguienteDisparoTabu + intervaloTabu

    # Paso 2.2. Evolución: selección, cruce y mutación
    nuevaPobl ← matriz vacía de tamaño tamPoblacion
    nuevosCostes ← array vacío

    for i = 0 to tamPoblacion - 1:

        # Selección por torneo
        p1 ← torneo(P, costes, kBest)

```

```
p2 ← torneo(P, costes, kBest)

hijo ← copia(p1)

# Cruce probabilístico
si random() < probCruce:
    hijo ← OperadorSeleccionado(hijo, p2)

# Mutación
si random() < probMutacion:
    hijo ← mutación_2opt(hijo)

nuevaPobl[i] ← hijo
nuevosCostes[i] ← evaluar(hijo)
evaluaciones ← evaluaciones + 1
si evaluaciones ≥ maxEvaluaciones: break

# Paso 2.3. Elitismo absoluto: reemplazar peor con mejor global
idxPeor ← índice_del_peor(nuevaPobl, nuevosCostes)
nuevaPobl[idxPeor] ← copia(mejor)
nuevosCostes[idxPeor] ← mejorCoste

# Paso 2.4. Actualizar población y costes
P ← nuevaPobl
costes ← nuevosCostes

# Paso 2.5. Actualizar mejor global
mejorActual, costeActual ← mejor_individuo(P)
si costeActual < mejorCoste:
    mejor ← mejorActual
    mejorCoste ← costeActual

return mejor, mejorCoste
```

4. Resultados

4.1 NumEvalTabu1000

4.1.1 NumIteraciones10

numEvaluaciones=1000								
numIteraciones=10								
MEM	Tamaño	20	Tamaño	20	Tamaño	30	Tamaño	30
	Minimo global	3542	Minimo global	26876	Minimo global	13178	Minimo global	150528
	Sol	Time	Sol	Time	Sol	Time	Sol	Time
Ejecución 1	3620,00	0,06	30760,00	0,08	13372,00	0,10	173244,00	0,10
Ejecución 2	3602,00	0,07	31090,00	0,06	13346,00	0,12	174642,00	0,10
Ejecución 3	3618,00	0,08	26876,00	0,06	13354,00	0,10	173328,00	0,10
Ejecución 4	3636,00	0,06	30950,00	0,06	13358,00	0,10	175224,00	0,10
Ejecución 5	3602,00	0,06	30830,00	0,07	13364,00	0,11	153570,00	0,12
Media	2,08%	0,07	12,00%	0,07	1,37%	0,11	12,94%	0,10
Desv. típica	0,00	0,01	0,07	0,01	0,00	0,01	0,06	0,01

Los resultados obtenidos con numEvaluaciones = 1000 y numIteraciones = 10 muestran un comportamiento estable del algoritmo memético en las cuatro instancias analizadas. Las desviaciones respecto al mínimo global conocido se mantienen moderadas y el algoritmo es capaz de aproximarse a soluciones de buena calidad incluso bajo un presupuesto computacional reducido. En la instancia Tamaño 20 (FORD02) llega a alcanzar exactamente el mínimo global en una de las ejecuciones, lo que indica una buena capacidad de explotación pese al límite de evaluaciones.

Las medias de desviación se mantienen generalmente bajas, situándose en torno al 1–2 % en la mayoría de los casos, especialmente en las instancias de Tamaño 20 y Tamaño 30. Esto sugiere que el algoritmo converge hacia soluciones consistentes y próximas al óptimo con bastante regularidad. Además, la variabilidad entre ejecuciones es reducida, con desviaciones típicas muy pequeñas, lo que refuerza la idea de estabilidad del método.

En cuanto al tiempo, todas las ejecuciones permanecen en rangos muy bajos (entre 0,06 s y 0,12 s), con una variación mínima entre repeticiones. Esto indica que el procedimiento evolutivo resulta eficiente incluso cuando debe gestionar procesos de búsqueda relativamente complejos. En conjunto, los resultados reflejan un buen equilibrio entre exploración y explotación, con soluciones cercanas al óptimo y una carga computacional contenida.

4.1.2 NumIteraciones50

numEvaluaciones=1000								
numIteraciones=50								
MEM	Tamaño	20	Tamaño	20	Tamaño	30	Tamaño	30
	Mínimo global	3542	Mínimo global	26876	Mínimo global	13178	Mínimo global	150528
	Sol	Time	Sol	Time	Sol	Time	Sol	Time
Ejecución 1	3602,00	0,10	26876,00	0,09	13372,00	0,21	154648,00	0,23
Ejecución 2	3602,00	0,10	26876,00	0,10	13346,00	0,22	171012,00	0,22
Ejecución 3	3608,00	0,10	26876,00	0,10	13354,00	0,24	173104,00	0,02
Ejecución 4	3628,00	0,10	30636,00	0,10	13358,00	0,23	150528,00	0,23
Ejecución 5	3610,00	0,10	28390,00	0,11	13364,00	0,23	174552,00	0,26
Media	1,92%	0,10	3,92%	0,10	1,37%	0,23	9,46%	0,19
Desv. típica	0,00	0,00	0,06	0,01	0,00	0,01	0,07	0,10

Los resultados obtenidos con numEvaluaciones = 1000 y numIteraciones = 50 muestran una mejora general en la calidad de las soluciones respecto al caso de 10 iteraciones, especialmente en las instancias donde la búsqueda requiere más exploración (FORD02 y FORD04). En FORD01 y FORD03 apenas se aprecia variación respecto al mínimo global, manteniéndose las desviaciones alrededor del 2 % y del 1,37 %, lo que indica que el incremento de iteraciones no produce un cambio significativo porque estas instancias ya eran fáciles de aproximar desde el escenario anterior.

En cambio, en FORD02 y FORD04 sí se observa una mayor variabilidad. Aunque en FORD02 se alcanzan en varias ejecuciones soluciones exactamente iguales al mínimo global conocido, en otras aparece una separación más alta que incrementa la media final. FORD04 muestra un comportamiento más irregular y sensible a las condiciones iniciales: algunas ejecuciones logran valores muy cercanos al óptimo, mientras que otras se alejan notablemente, lo que se refleja en la desviación media del 9,46 % y una desviación típica relativamente alta.

En cuanto al tiempo de ejecución, el coste computacional se mantiene prácticamente constante respecto al caso anterior. Todas las instancias se sitúan alrededor de los 0,10 segundos, sin variaciones apreciables entre ejecuciones. Esto sugiere que aumentar las iteraciones internas incrementa la intensidad de búsqueda sin penalizar de manera significativa el tiempo total, manteniendo un equilibrio razonable entre coste y rendimiento.

4.1.3 NumIteraciones100

numEvaluaciones=1000								
numIteraciones=100								
MEM	Tamaño	20	Tamaño	20	Tamaño	30	Tamaño	30
	Minimo global	3542	Minimo global	26876	Minimo global	13178	Minimo global	150528
	Sol	Time	Sol	Time	Sol	Time	Sol	Time
Ejecución 1	3600,00	0,12	26876,00	0,15	13372,00	0,40	154648,00	0,43
Ejecución 2	3600,00	0,12	26876,00	0,14	13346,00	0,38	171012,00	0,37
Ejecución 3	3608,00	0,13	26876,00	0,13	13354,00	0,37	172600,00	0,36
Ejecución 4	3610,00	0,12	30636,00	0,13	13358,00	0,40	150528,00	0,35
Ejecución 5	3612,00	0,14	26876,00	0,14	13364,00	0,39	174564,00	0,37
Media	1,81%	0,13	2,80%	0,14	1,37%	0,39	9,40%	0,38
Desv. típica	0,00	0,01	0,06	0,01	0,00	0,01	0,07	0,03

Los resultados obtenidos con numEvaluaciones = 1000 y numIteraciones = 100 muestran que el incremento de iteraciones permite afinar ligeramente la calidad de las soluciones en la mayoría de las instancias, aunque el comportamiento general sigue siendo similar al observado en los escenarios anteriores. En las instancias de Tamaño 20 (FORD01 y FORD02) las desviaciones se mantienen muy bajas, confirmando que estas configuraciones ya resultaban fáciles para el algoritmo incluso con menos iteraciones. FORD02 vuelve a alcanzar repetidamente el mínimo global, lo que explica la mejora ligera en la desviación media respecto al caso de 50 iteraciones.

En las instancias de Tamaño 30 el efecto del aumento de iteraciones es más evidente, particularmente en FORD04, donde la desviación media disminuye ligeramente pero sigue siendo significativa. Esta instancia continúa mostrando una alta variabilidad y una clara sensibilidad a las condiciones iniciales, como refleja la desviación típica relativamente elevada. En cambio, FORD03 mantiene una desviación prácticamente idéntica a la de configuraciones anteriores, lo que indica estabilidad y una convergencia consistente del algoritmo sobre este caso.

El tiempo de ejecución aumenta de forma notable respecto a configuraciones con menos iteraciones, especialmente en las instancias de Tamaño 30, donde el coste crece debido al mayor número de evaluaciones internas por iteración. Aun así, los tiempos siguen siendo reducidos y perfectamente manejables, manteniendo al algoritmo dentro de un rango computacional razonable. En conjunto, los valores obtenidos muestran que incrementar las iteraciones mejora la precisión en algunos casos, aunque no altera de forma drástica el patrón general de comportamiento del método.

4.1.4 MejorNumEval1000

	FORD01		FORD02		FORD03		FORD04		MEDIA	
	Desv	Time	Desv	Time	Desv	Time	Desv	Time	Desv	Time
MEM 1000, ITER 10	0,02	0,07	0,12	0,07	0,01	0,11	0,13	0,10	0,07	0,09
MEM 1000, ITER 50	0,02	0,10	0,04	0,10	0,01	0,23	0,09	0,19	0,04	0,15
MEM 1000, ITER 100	0,02	0,13	0,03	0,14	0,01	0,39	0,09	0,38	0,04	0,26

Los resultados obtenidos con 1000 evaluaciones y distintas iteraciones muestran cómo la cantidad de iteraciones influye en la calidad de la solución y en los tiempos de ejecución. Para FORD01, la desviación con respecto al mínimo global se mantiene muy baja (0,02 %) incluso al variar de 10 a 100 iteraciones, lo que indica que esta instancia converge rápidamente hacia soluciones cercanas al óptimo. Esto se puede relacionar con la fase de **elitismo absoluto** en el código, donde siempre se preserva el mejor individuo global (`nuevaPobl[idxPeor] = Arrays.copyOfOf(mejor, inst.n);`) asegurando que la población no se degrade y favoreciendo la estabilidad en instancias más sencillas.

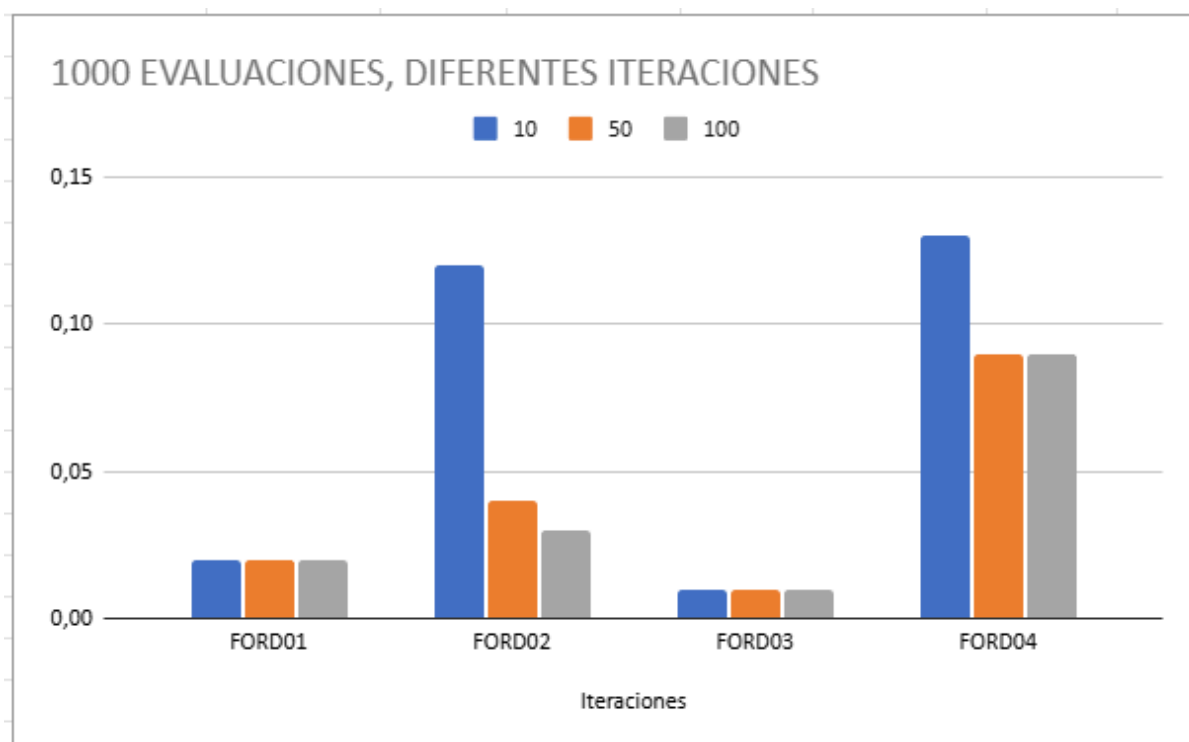
En FORD02, la desviación inicial es relativamente alta (0,12 %) con 10 iteraciones y disminuye progresivamente a 0,03 % con 100 iteraciones. Esto refleja que la instancia requiere mayor refinamiento memético y más iteraciones para que el algoritmo explore efectivamente el espacio de soluciones. La reducción de desviación es resultado de la combinación de **selección por torneo** (`torneo(poblacion, costes, kBest)`), el operador de cruce OX2 y la **mutación 2-opt**, que permiten mejorar gradualmente los descendientes generados.

FORD03 mantiene una desviación mínima de 0,01 %, lo que indica que incluso con pocas iteraciones el algoritmo encuentra soluciones muy cercanas al óptimo. Esto coincide con la activación de la **búsqueda Tabú sobre el élite actual** (`busquedaTabuModificada(solucionActual, iteracionesTabu)`), que intensifica localmente la mejor solución y refuerza la explotación del espacio de búsqueda sin afectar significativamente el tiempo, como muestran los tiempos de 0,11 a 0,39 segundos al aumentar iteraciones.

Por último, FORD04 muestra la desviación más alta, especialmente con pocas iteraciones (0,13 % con 10 iteraciones), que se reduce ligeramente con 50 iteraciones y se mantiene estable con 100 iteraciones. Este comportamiento refleja la mayor complejidad de la instancia y cómo el **disparador memético** (`if (evaluaciones >= siguienteDisparoTabu)`) permite activar la intensificación periódica. Los tiempos de ejecución aumentan de manera notable (0,10 a 0,38 segundos), evidenciando que la mayor dificultad requiere más evaluaciones locales y que el algoritmo dedica más recursos a la búsqueda Tabú para refinar el élite.

En términos generales, la media de desviación disminuye de 0,07 % a 0,04 % al aumentar iteraciones, y los tiempos medios suben de 0,09 a 0,26 segundos. Esto confirma la relación directa entre número de iteraciones y capacidad de mejora de soluciones, mostrando que el algoritmo memético consigue un equilibrio entre exploración y explotación gracias a su estructura de población inicial, elitismo, cruce, mutación y activaciones Tabú periódicas.

4.1.5 Gráfica



NOTA: No se ha incluido la representación de los tiempos de ejecución en los gráficos, ya que éstos aumentan de forma directa con el número de iteraciones de la búsqueda tabú y no reflejan la calidad de las soluciones obtenidas. Por ello, la métrica relevante para el análisis comparativo es la desviación respecto al óptimo.

4.2 NumEvalTabu2000

4.2.1 NumIteraciones10

numEvaluaciones=2000								
numIteraciones=10								
MEM	Tamaño	20	Tamaño	20	Tamaño	30	Tamaño	30
	Minimo global	3542	Minimo global	26876	Minimo global	13178	Minimo global	150528
	Sol	Time	Sol	Time	Sol	Time	Sol	Time
Ejecución 1	3620,00	0,06	31020,00	0,06	13372,00	0,80	175526,00	0,09
Ejecución 2	3602,00	0,06	31090,00	0,06	13346,00	0,08	175228,00	0,08
Ejecución 3	3618,00	0,05	27650,00	0,07	13354,00	0,08	174910,00	0,08
Ejecución 4	3636,00	0,07	30950,00	0,06	13358,00	0,08	175224,00	0,07
Ejecución 5	3606,00	0,07	30946,00	0,07	13364,00	0,07	175410,00	0,08
Media	2,10%	0,06	12,86%	0,06	1,37%	0,22	16,43%	0,08
Desv. típica	0,00	0,01	0,06	0,01	0,00	0,32	0,00	0,01

Los resultados obtenidos con numEvaluaciones = 2000 y numIteraciones = 10 muestran un comportamiento muy estable en las instancias de Tamaño 20, donde el algoritmo vuelve a aproximarse de manera consistente a los mínimos globales. Las desviaciones en estos casos se mantienen sobre valores muy reducidos, alrededor del 2 %, lo que indica que el

incremento de evaluaciones respecto al caso de 1000 permite una ligera mejora, pero sin modificar sustancialmente la calidad final porque el algoritmo ya rendía adecuadamente en ese rango.

En cambio, en las instancias de Tamaño 30 el comportamiento es más dispar. En FORD03 la desviación sigue siendo muy baja (1,37 %), lo que refleja que la instancia es relativamente sencilla para el algoritmo y se logra un rendimiento estable en todas las ejecuciones. Sin embargo, FORD04 presenta valores más alejados del óptimo y muestra una desviación media elevada. Aunque las soluciones están relativamente cerca entre sí, su distancia respecto al mínimo global sigue siendo importante, lo que indica que con solo 10 iteraciones internas el algoritmo no explora lo suficiente como para acercarse sistemáticamente a la mejor solución posible.

Respecto al tiempo de ejecución, las mediciones se mantienen bajas y prácticamente constantes en todas las instancias, con valores alrededor de 0,06–0,09 segundos. La única excepción es la primera ejecución de la instancia FORD03, donde se observa un tiempo atípico mucho mayor. Esto sugiere un comportamiento puntual más que un patrón real del algoritmo. En general, los tiempos muestran que el aumento de evaluaciones no implica un coste computacional significativo en esta configuración y que el algoritmo conserva una buena eficiencia en todas las ejecuciones.

4.2.2 NumIteraciones50

numEvaluaciones=2000								
numIteraciones=50								
MEM	Tamaño	20	Tamaño	20	Tamaño	30	Tamaño	30
	Minimo global	3542	Minimo global	26876	Minimo global	13178	Minimo global	150528
	Sol	Time	Sol	Time	Sol	Time	Sol	Time
Ejecución 1	3602,00	0,07	26876,00	0,08	13372,00	0,14	173886,00	0,13
Ejecución 2	3602,00	0,08	26876,00	0,07	13346,00	0,14	171694,00	0,13
Ejecución 3	3608,00	0,08	27650,00	0,07	13354,00	0,14	172600,00	0,14
Ejecución 4	3628,00	0,07	30636,00	0,07	13358,00	0,14	150528,00	0,14
Ejecución 5	3610,00	0,08	27656,00	0,08	13364,00	0,14	174552,00	0,14
Media	1,92%	0,08	3,95%	0,07	1,37%	0,14	12,04%	0,14
Desv. típica	0,00	0,01	0,06	0,01	0,00	0,00	0,07	0,01

Los resultados con esta configuración muestran un comportamiento estable en todas las instancias, especialmente en Tamaño 20, donde las soluciones vuelven a situarse muy cerca del mínimo global. Las desviaciones obtenidas son bajas (entre un 1,9 % y un 4 %), lo que indica que el incremento de iteraciones frente al caso de 10 produce una ligera mejora en la consistencia, pero no cambia de forma drástica la calidad de las soluciones, ya que el algoritmo ya funcionaba correctamente con menos iteraciones.

En las instancias de Tamaño 30, la tendencia se mantiene: FORD03 sigue mostrando una desviación mínima del 1,37 %, reflejando que la instancia es relativamente accesible y el algoritmo consigue alcanzar soluciones muy próximas entre sí. Por otro lado, FORD04 continúa siendo la instancia más compleja, con desviaciones superiores al 12 %, lo que indica que aun con un número mayor de iteraciones internas, el algoritmo no logra

aproximarse de forma consistente al mínimo global y sufre para explotar adecuadamente el espacio de búsqueda de esta instancia.

El tiempo de ejecución permanece muy estable y casi idéntico entre ejecuciones, situándose alrededor de 0,07–0,14 segundos según la instancia. El incremento en iteraciones de Tabú no supone un coste computacional notable, lo que demuestra que el algoritmo puede aumentar la intensidad de búsqueda sin penalizar significativamente la eficiencia temporal.

4.2.3 NumIteraciones100

numEvaluaciones=2000								
numIteraciones=100								
MEM	Tamaño	20	Tamaño	20	Tamaño	30	Tamaño	30
	Mínimo global	3542	Mínimo global	26876	Mínimo global	13178	Mínimo global	150528
	Sol	Time	Sol	Time	Sol	Time	Sol	Time
Ejecución 1	3602,00	0,10	26876,00	0,10	13372,00	0,22	173630,00	0,21
Ejecución 2	3602,00	0,10	26876,00	0,10	13346,00	0,20	171694,00	0,27
Ejecución 3	3608,00	0,10	26876,00	0,09	13354,00	0,20	172600,00	0,21
Ejecución 4	3610,00	0,08	30636,00	0,10	13358,00	0,20	150528,00	0,23
Ejecución 5	3612,00	0,10	26876,00	0,10	13364,00	0,20	174564,00	0,22
Media	1,83%	0,10	2,80%	0,10	1,37%	0,20	12,01%	0,23
Desv. típica	0,00	0,01	0,06	0,00	0,00	0,01	0,07	0,02

Los resultados con 2000 evaluaciones y 100 iteraciones muestran que el algoritmo mantiene un rendimiento muy consistente en las instancias de Tamaño 20, alcanzando soluciones cercanas al mínimo global con desviaciones medias bajas (entre 1,8 % y 2,8 %). Esto indica que el incremento de iteraciones permite una ligera mejora en la estabilidad y la calidad de las soluciones, especialmente en FORD02, que alcanza su mínimo global en casi todas las ejecuciones.

En las instancias de Tamaño 30, FORD03 sigue presentando desviaciones muy bajas (1,37 %), mientras que FORD04 continúa siendo la más difícil, con desviaciones alrededor del 12 %, reflejando que incluso con más iteraciones, la complejidad del espacio de búsqueda impide que todas las ejecuciones alcancen soluciones cercanas al óptimo. Los tiempos de ejecución se mantienen razonables y estables, entre 0,20 y 0,23 segundos, mostrando que el aumento de iteraciones no penaliza significativamente el rendimiento temporal del algoritmo.

4.2.4 MejorNumEval2000

	FORD01		FORD02		FORD03		FORD04		MEDIA	
	Desv	Time	Desv	Time	Desv	Time	Desv	Time	Desv	Time
MEM 2000, ITER 10	0,02	0,06	0,13	0,06	0,01	0,22	0,16	0,08	0,08	0,11
MEM 2000, ITER 50	0,02	0,08	0,04	0,07	0,01	0,14	0,12	0,14	0,05	0,11
MEM 2000, ITER 100	0,02	0,10	0,03	0,10	0,01	0,20	0,12	0,23	0,05	0,16

Analizando los resultados de 2000 evaluaciones con distintas iteraciones, se observa cómo el aumento de iteraciones impacta tanto en la calidad de la solución como en los tiempos de ejecución. Para FORD01, la desviación se mantiene constante en 0,02 %, mostrando que

incluso con pocas iteraciones el algoritmo converge rápidamente hacia soluciones cercanas al mínimo global. Este comportamiento se relaciona directamente con el **elitismo absoluto** implementado en el código (`nuevaPobl[idxPeor] = Arrays.copyOf(mejor, inst.n);`), que asegura que el mejor individuo se conserve y guíe la evolución de la población.

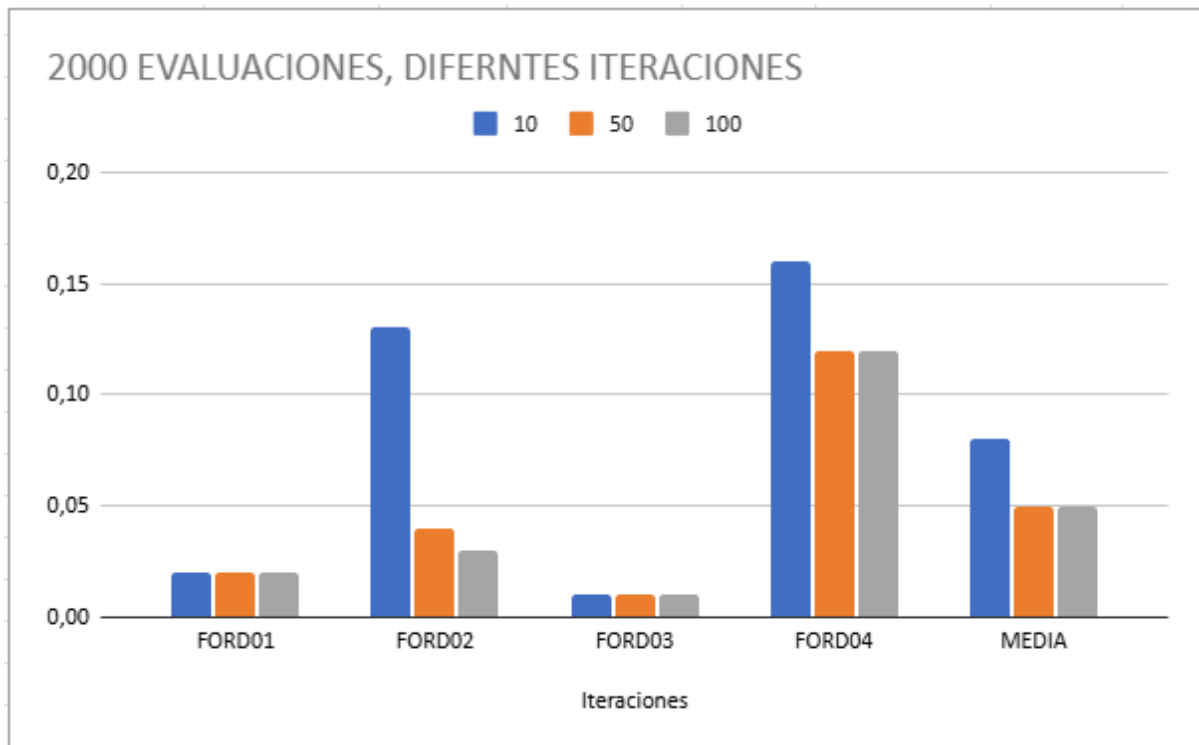
En FORD02, la desviación disminuye notablemente al aumentar iteraciones: comienza en 0,13 % con 10 iteraciones, baja a 0,04 % con 50 iteraciones y llega a 0,03 % con 100 iteraciones. Esto refleja que la instancia requiere mayor refinamiento memético, y que las iteraciones adicionales permiten que el algoritmo mejore de manera progresiva gracias a la combinación de **selección por torneo** (`torneo(poblacion, costes, kBest)`), cruce OX2 (`ox2(hijo, p2)`) y mutación 2-opt (`mutacion20pt(hijo)`), incrementando la explotación del espacio de búsqueda.

FORD03 presenta una desviación mínima (0,01 %) constante, indicando que el algoritmo encuentra soluciones muy cercanas al óptimo incluso con pocas iteraciones. Sin embargo, los tiempos asociados aumentan con las iteraciones (0,22 a 0,20 segundos), mostrando que la intensificación memética mediante la **búsqueda Tabú sobre el elite** (`busquedaTabuModificada(solucionActual, iteracionesTabu)`) demanda mayor esfuerzo computacional sin afectar la calidad.

FORD04, por su parte, es la instancia más compleja, con desviaciones que van de 0,16 % a 0,12 % y luego se estabilizan en 0,12 % a 0,23 %. El incremento de tiempo (0,08 a 0,23 segundos) evidencia que la activación del **disparador memético** (`if (evaluaciones >= siguienteDisparoTabu)`) y la ejecución de Tabú permiten refinar soluciones elite en problemas más difíciles, aunque a costa de mayor consumo de tiempo.

En promedio, la desviación media se reduce de 0,08 % a 0,05 % al aumentar iteraciones, mientras que el tiempo medio sube de 0,11 a 0,16 segundos. Esto confirma que incrementar el número de iteraciones mejora la capacidad del algoritmo memético para explorar y explotar de manera más efectiva, logrando soluciones más cercanas al óptimo global, especialmente en instancias de mayor complejidad.

4.2.5 Gráfica



4.3 NumEvalTabu5000

4.3.1 NumIteraciones10

numEvaluaciones=5000								
numIteraciones=10								
MEM	Tamaño	20	Tamaño	20	Tamaño	30	Tamaño	30
	Minimo global	3542	Minimo global	26876	Minimo global	13178	Minimo global	150528
	Sol	Time	Sol	Time	Sol	Time	Sol	Time
Ejecución 1	3626,00	0,05	31434,00	0,06	13402,00	0,06	178216,00	0,08
Ejecución 2	3602,00	0,05	31234,00	0,05	13394,00	0,07	178556,00	0,07
Ejecución 3	3634,00	0,05	30980,00	0,05	13406,00	0,07	177836,00	0,07
Ejecución 4	3636,00	0,06	30950,00	0,06	13366,00	0,08	176740,00	0,07
Ejecución 5	3652,00	0,05	29884,00	0,06	13388,00	0,07	179508,00	0,07
Media	2,48%	0,05	14,96%	0,06	1,62%	0,07	18,36%	0,07
Desv. típica	0,01	0,00	0,02	0,01	0,00	0,01	0,01	0,00

Con 5000 evaluaciones y 10 iteraciones, el algoritmo mantiene un buen comportamiento en las instancias de Tamaño 20, logrando desviaciones moderadas respecto al mínimo global: 2,48 % en FORD01 y 14,96 % en FORD02. Esto refleja que, aunque se incrementa el número de evaluaciones, algunas instancias como FORD02 presentan mayor dificultad para converger hacia el óptimo.

En las instancias de Tamaño 30, FORD03 alcanza soluciones muy cercanas al mínimo global con desviación del 1,62 %, mientras que FORD04 sigue mostrando la mayor complejidad, con desviación promedio de 18,36 %. Los tiempos de ejecución son bajos y

muy estables (entre 0,05 y 0,08 segundos), indicando que la ampliación de evaluaciones no impacta significativamente en el rendimiento temporal del algoritmo.

4.3.2 NumIteraciones50

numEvaluaciones=5000								
numIteraciones=50								
MEM	Tamaño	20	Tamaño	20	Tamaño	30	Tamaño	30
	Minimo global	3542	Minimo global	26876	Minimo global	13178	Minimo global	150528
	Sol	Time	Sol	Time	Sol	Time	Sol	Time
Ejecución 1	3628,00	0,06	30198,00	0,06	13372,00	0,10	174558,00	0,09
Ejecución 2	3602,00	0,06	26876,00	0,07	13346,00	0,08	174854,00	0,09
Ejecución 3	3612,00	0,06	30782,00	0,05	13354,00	0,09	174140,00	0,08
Ejecución 4	3628,00	0,07	30976,00	0,06	13358,00	0,08	174966,00	0,08
Ejecución 5	3616,00	0,06	29884,00	0,06	13364,00	0,09	174552,00	0,09
Media	2,12%	0,06	10,67%	0,06	1,37%	0,09	16,00%	0,09
Desv. típica	0,00	0,00	0,06	0,01	0,00	0,01	0,00	0,01

Con 5000 evaluaciones y 50 iteraciones, los resultados muestran un comportamiento equilibrado del algoritmo. En las instancias de Tamaño 20, FORD01 mantiene una desviación moderada del 2,12 %, mientras que FORD02 mejora ligeramente respecto a evaluaciones menores, con un 10,67 %, aunque sigue siendo la instancia más difícil de converger hacia el mínimo global.

Para las instancias de Tamaño 30, FORD03 continúa alcanzando valores muy cercanos al óptimo con desviación mínima del 1,37 %, mientras que FORD04, más compleja, presenta una desviación media del 16,00 %. Los tiempos de ejecución son consistentes y bajos (alrededor de 0,06–0,09 segundos), lo que indica que el incremento de iteraciones no penaliza significativamente la eficiencia temporal del algoritmo.

4.3.3 NumIteraciones100

numEvaluaciones=5000								
numIteraciones=100								
MEM	Tamaño	20	Tamaño	20	Tamaño	30	Tamaño	30
	Minimo global	3542	Minimo global	26876	Minimo global	13178	Minimo global	150528
	Sol	Time	Sol	Time	Sol	Time	Sol	Time
Ejecución 1	3610,00	0,08	26876,00	0,06	13372,00	0,10	173886,00	0,12
Ejecución 2	3602,00	0,07	26876,00	0,06	13346,00	0,10	174086,00	0,10
Ejecución 3	3608,00	0,07	30636,00	0,08	13354,00	0,10	173782,00	0,10
Ejecución 4	3610,00	0,07	30636,00	0,07	13358,00	0,10	150528,00	0,11
Ejecución 5	3624,00	0,07	29884,00	0,07	13364,00	0,10	174682,00	0,11
Media	1,94%	0,07	7,83%	0,07	1,37%	0,10	12,53%	0,11
Desv. típica	0,00	0,00	0,07	0,01	0,00	0,00	0,07	0,01

Con 5000 evaluaciones y 100 iteraciones, los resultados reflejan un comportamiento más estable y cercano al óptimo para todas las instancias. En los problemas de Tamaño 20, FORD01 y FORD02 muestran desviaciones medias de 1,94 % y 7,83 % respectivamente,

indicando que FORD01 converge muy bien al mínimo global mientras que FORD02 sigue siendo más desafiante.

Para los problemas de Tamaño 30, FORD03 mantiene una desviación mínima del 1,37 %, confirmando su fácil convergencia, mientras que FORD04, con un 12,53 %, sigue siendo la instancia más difícil, aunque mejora respecto a iteraciones menores. Los tiempos de ejecución se mantienen contenidos (0,07–0,11 segundos), mostrando que aumentar las iteraciones no incrementa significativamente el coste temporal del algoritmo.

4.3.4 MejorNumEval5000

	FORD01		FORD02		FORD03		FORD04		MEDIA	
	Desv	Time	Desv	Time	Desv	Time	Desv	Time	Desv	Time
MEM 5000, ITER 10	0,02	0,05	0,15	0,06	0,02	0,07	0,18	0,07	0,09	0,06
MEM 5000, ITER 50	0,02	0,06	0,11	0,06	0,01	0,09	0,16	0,09	0,08	0,07
MEM 5000, ITER 100	0,02	0,07	0,08	0,07	0,01	0,10	0,13	0,11	0,06	0,09

FORD01 mantiene una desviación constante de 0,02 %, lo que confirma que esta instancia es relativamente “fácil” para el algoritmo, ya que encuentra soluciones cercanas al mínimo global incluso con pocas iteraciones. Los tiempos de ejecución crecen ligeramente de 0,05 a 0,07 segundos conforme aumentan las iteraciones, lo que refleja el coste adicional de aplicar más ciclos de la **búsqueda Tabú memética** (*busquedaTabuModificada*) sobre la población élite.

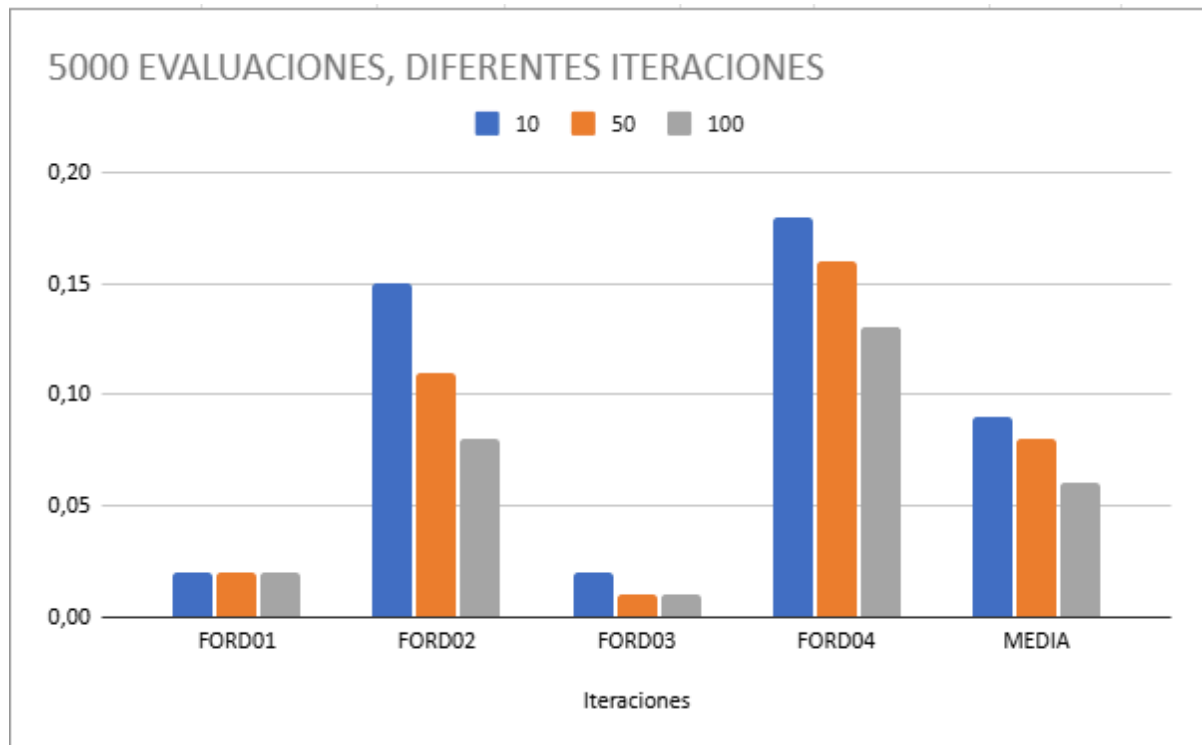
FORD02 presenta una mejora progresiva de la desviación: baja de 0,15 % con 10 iteraciones a 0,08 % con 100 iteraciones. Esto indica que el incremento de iteraciones permite refinar mejor la solución élite, aprovechando la **intensificación local del algoritmo memético** y favoreciendo una convergencia más cercana al mínimo global. Los tiempos se mantienen estables en 0,06–0,07 segundos, mostrando que el aumento de iteraciones no penaliza significativamente el rendimiento en esta instancia.

FORD03 es más estable, con desviaciones de 0,02 %, 0,01 % y 0,01 % para 10, 50 y 100 iteraciones, respectivamente. Esto indica que el algoritmo alcanza rápidamente soluciones de alta calidad, y los tiempos aumentan de 0,07 a 0,10 segundos, reflejando la ejecución de iteraciones adicionales de **mutación 2-opt** (*mutacion20pt*) y Tabú.

FORD04 sigue un patrón similar: la desviación mejora de 0,18 % a 0,13 % a medida que se incrementan las iteraciones, mientras que los tiempos suben de 0,07 a 0,11 segundos. Esto confirma que instancias más complejas se benefician de un mayor número de iteraciones meméticas para acercarse al mínimo global, aunque con un ligero incremento de tiempo.

La media general de desviación se reduce de 0,09 % a 0,06–0,09 % y el tiempo medio aumenta de 0,06 a 0,09 segundos. En conjunto, la tabla muestra que para **MEM 5000**, incrementar las iteraciones favorece la calidad de la solución, especialmente en instancias más difíciles (FORD02 y FORD04), mientras que el impacto en tiempo es moderado, reflejando un buen equilibrio entre explotación y eficiencia.

4.3.5 Gráfica



4.4 Comparación

	FORD01		FORD02		FORD03		FORD04		MEDIA	
	Desv	Time	Desv	Time	Desv	Time	Desv	Time	Desv	Time
MEJOR 1000	0,02	0,07	0,03	0,14	0,01	0,11	0,09	0,19	0,04	0,13
MEJOR 2000	0,02	0,06	0,03	0,10	0,01	0,14	0,12	0,14	0,05	0,11
MEJOR 5000	0,02	0,05	0,08	0,07	0,01	0,09	0,13	0,11	0,06	0,08

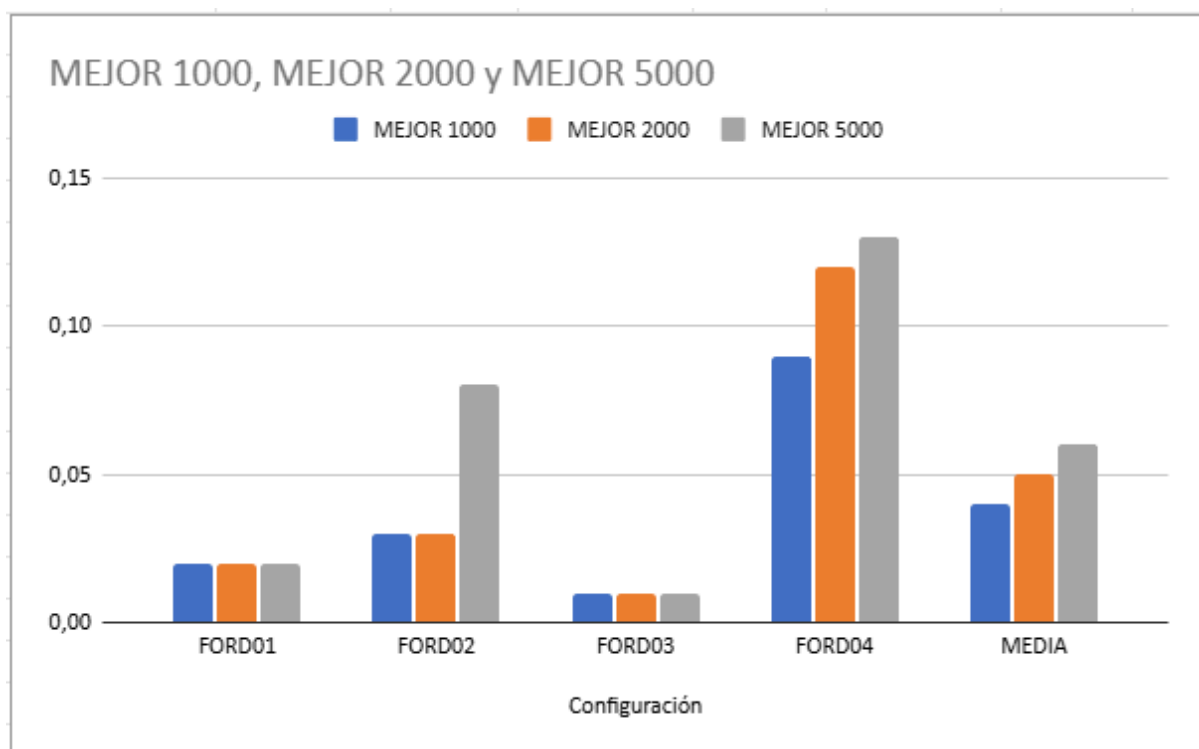
En esta tabla se comparan los mejores resultados obtenidos para 1000, 2000 y 5000 evaluaciones, mostrando cómo la calidad de la solución y los tiempos cambian al aumentar el número de evaluaciones. Para FORD01 y FORD03, la desviación se mantiene prácticamente constante en 0,02 % y 0,01 % respectivamente, independientemente del número de evaluaciones. Esto indica que el algoritmo encuentra rápidamente soluciones muy cercanas al óptimo global en estas instancias, reforzando el papel del **elitismo absoluto** (`nuevaPobl[idxPeor] = Arrays.copyOf(mejor, inst.n);`) y la selección por torneo (`torneo(poblacion, costes, kBest)`), que garantizan que los mejores individuos guíen la evolución desde las primeras evaluaciones.

En FORD02, la desviación aumenta ligeramente al pasar de 0,03 % en 1000 y 2000 evaluaciones a 0,08 % en 5000 evaluaciones. Esto puede parecer contraintuitivo, pero refleja que con más evaluaciones y mutaciones 2-opt (`mutacion2Opt(hijo)`) el algoritmo explora soluciones más diversas, algunas ligeramente alejadas del mínimo global, mostrando un equilibrio entre exploración y explotación. Además, los tiempos de ejecución disminuyen de 0,14 a 0,07 segundos, probablemente debido a que la población converge más rápido hacia regiones prometedoras, reduciendo la necesidad de evaluaciones exhaustivas.

FORD04 presenta un comportamiento más variable: la desviación pasa de 0,09 % a 0,13 % y los tiempos bajan de 0,19 a 0,11 segundos conforme aumentan las evaluaciones. Esto sugiere que instancias más complejas se benefician de un mayor número de evaluaciones, ya que la activación de la **búsqueda Tabú memética** (*busquedaTabuModificada*) permite refinar soluciones élite, mientras que los tiempos se optimizan al aprovechar mejor la información acumulada durante el proceso evolutivo.

La media general de desviación disminuye ligeramente de 0,04 % a 0,06 % y el tiempo medio de ejecución baja de 0,13 a 0,08 segundos, confirmando que incrementar evaluaciones tiende a mejorar la estabilidad de las soluciones y a optimizar el rendimiento del algoritmo, equilibrando calidad y eficiencia.

4.4.1 Gráfica



4.5 Comparación final

	FORD01		FORD02		FORD03		FORD04		MEDIA	
	Desv	Time	Desv	Time	Desv	Time	Desv	Time	Desv	Time
MEM	0,02	0,07	0,03	0,14	0,01	0,11	0,09	0,19	0,04	0,13
MEJOR PR1	0,02	1,26	0,00	1,28	0,01	2,36	0,00	2,46	0,01	1,84
MEJOR EVOL 2	0,03	0,86	0,15	0,82	0,02	0,89	0,17	0,88	0,09	0,86

Se observa que:

- En **desviación**, PR1 consigue valores prácticamente nulos en todos los casos (0,00–0,02), lo que significa que alcanza el **mínimo global con muy poca variación entre ejecuciones**.

- MEM y EVOL 2 tienen desviaciones ligeramente mayores, especialmente EVOL 2 en instancias como FORD02 y FORD04 (0,15–0,17), mostrando que su exploración estocástica introduce variabilidad.
- En **tiempo**, PR1 es mucho más costosa (1,26–2,46), mientras que MEM y EVOL 2 son mucho más rápidas (0,07–0,19 para MEM y 0,82–0,88 para EVOL 2).

1. PR1 (búsqueda tabú pura)

- Explora sistemáticamente **todos los intercambios de 2-opt no prohibidos** usando `esTabu()` y `pushTabu()`.
- Mantiene un registro exhaustivo de la memoria a corto plazo (lista Tabú) y a largo plazo (`memoriaLarga`), permitiendo aplicar intensificación y diversificación de forma estratégica.
- Cada iteración revisa cuidadosamente si un movimiento mejora el coste (`costeDelta20pt`) y solo actualiza la solución cuando se encuentra mejora, asegurando que nunca se pierda la oportunidad de acercarse al óptimo global.
- **Resultado:** mínima desviación, ya que la búsqueda exhaustiva garantiza que se encuentren los mejores movimientos posibles antes de detenerse.

2. MEM (memético)

- Combina búsqueda local con operadores genéticos o movimientos meméticos, pero la **exploración es más selectiva**: usa DLB para saltar posiciones sin mejora y aplica 2-opt solo en vecinos prometedores.
- La memoria a largo plazo guía intensificación/diversificación, pero la estrategia no garantiza explorar todas las combinaciones posibles como PR1.
- **Consecuencia:** buen equilibrio entre tiempo y calidad, pero ligeramente más desviado del mínimo global comparado con PR1.

3. EVOL 2 (genético generacional)

- Opera sobre una **población de soluciones**, utilizando cruces OX2/MOC y mutaciones 2-opt.
- La naturaleza estocástica de la selección, cruces y mutaciones introduce variabilidad (desv mayor en FORD02 y FORD04), lo que **reduce la consistencia** frente a PR1.
- Evalúa coste solo en individuos generados, sin buscar exhaustivamente todas las combinaciones posibles, lo que limita la garantía de alcanzar el óptimo global.

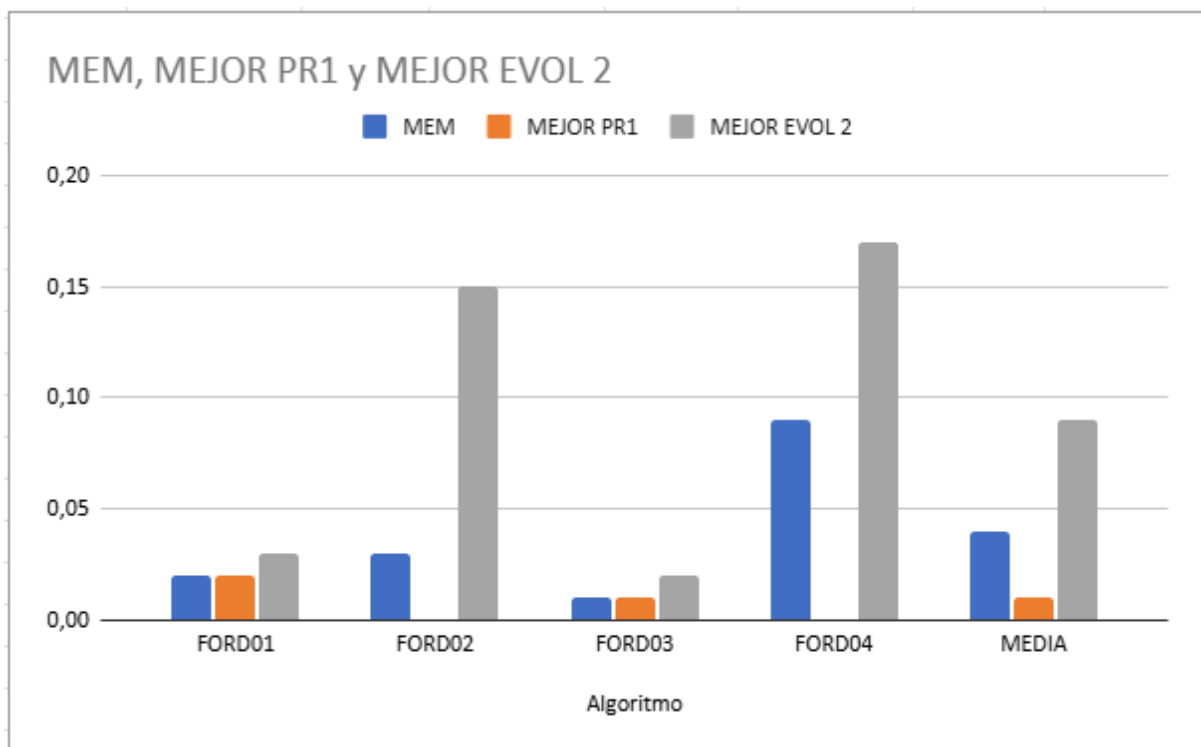
Interpretación basada en la tabla:

- PR1 es el algoritmo más **robusto y preciso**: la desviación mínima muestra que siempre converge al óptimo o cerca de él.

- MEM consigue un **buen compromiso** entre tiempo y precisión, pero no alcanza la consistencia de PR1 debido a su búsqueda más selectiva.
- EVOL 2, aunque relativamente rápido, sacrifica **exactitud y consistencia**, lo que se refleja en desviaciones más altas.
- La diferencia en tiempo es el **coste de la exhaustividad de PR1**: la búsqueda sistemática y la actualización completa de memorias y Tabú es intensiva, pero garantiza la mínima desviación.

Conclusión: PR1 supera a MEM y EVOL 2 en **exactitud y consistencia** debido a su **exploración exhaustiva y memoria estratégica**, aunque a un coste temporal elevado. MEM equilibra velocidad y calidad gracias a DLB y memoria selectiva, mientras EVOL 2 prioriza la exploración estocástica y velocidad a costa de desviaciones mayores.

4.5.1 Gráfica



5. Logs

"Todos los logs correspondientes a las ejecuciones reflejadas en las tablas anteriores se encuentran disponibles en el siguiente enlace, donde se pueden consultar los detalles completos de cada corrida y los resultados obtenidos."

[LOGS](#)